

Université Paris Cité

Étude du groupe de Lie $SO(n)$ dans le cadre de l'analyse de formes

Rapport de stage recherche

Antonin Lecocq

Sous la direction de Rayane Mouhli

Laboratoire de Mathématiques Appliquées à Paris 5
(MAP5, CNRS UMR 8145)

Licence 2 Mathématiques-Informatique

Année universitaire 2025–2026

Description du stage

Contexte Scientifique

L'analyse de formes est un domaine à l'intersection de la géométrie différentielle, de la mécanique géométrique et du traitement d'image. Son objectif principal est de quantifier et de modéliser les déformations de formes telles que des images, courbes, surfaces, etc. [7]. Un problème qui nous intéresse particulièrement est celui du recalage, c'est-à-dire, étant donné une forme source et une forme cible, comment modéliser la déformation optimale permettant de transformer l'une en l'autre.

Une méthode classique pour traiter ce problème est le cadre *Large Deformation Diffeomorphic Metric Mapping* (LDDMM, [1]), qui modélise les déformations comme des flots générés par des champs de vecteurs. Ce stage propose d'abord les bases de cette théorie en s'intéressant au cas particulier des rotations, modélisées par le groupe $SO(n)$. Ce groupe est fondamental dans l'analyse des formes car avec les translations, il permet de modéliser l'entière des déformations possibles pour des corps solides [2].

Objectifs du stage

- Revue de la littérature pour comprendre les bases du modèle *Large Deformation Diffeomorphic Metric Mapping* (LDDMM).
- Étude du groupe de rotations $SO(n)$ et de son algèbre de Lie $\mathfrak{so}(n)$.
- Étude de l'équation différentielle linéaire $\dot{x}(t) = Ax(t)$ avec $A \in \mathfrak{so}(n)$.
- Cas particuliers de $SO(2)$ et $SO(3)$.
- Implémentation en Python de modèles de recalage avec $SO(n)$.

Table des matières

Description du stage	1
Introduction	2
1 Le modèle de recalage LDDMM	3
1.1 Déformations et Difféomorphismes	3
1.2 Problème variationnel LDDMM	4
2 Le groupe $SO(n)$ et son algèbre de Lie $\mathfrak{so}(n)$	5
2.1 Le groupe de Lie $SO(n)$	5
2.1.1 Propriétés de l'action du groupe $SO(n)$ sur $\mathbb{R}^n$	7
2.1.2 Propriétés topologiques de $SO(n)$	9
2.2 L'algèbre de Lie $\mathfrak{so}(n)$	10
2.3 L'application exponentielle	13
3 Étude de l'équation différentielle linéaire $\dot{x}(t) = Ax(t)$	15
3.1 Obtenir l'équation différentielle $\dot{x}(t) = Ax(t)$	15
3.2 Solution de $\dot{x}(t) = Ax(t)$	16
4 Cas particuliers de $SO(2)$ et $SO(3)$	18
4.1 $SO(2)$, Les rotations en dimension 2	18
5 Implémentation algorithmique	21
5.1 Algorithme de résolution - Discrétiser puis Optimiser	21
5.1.1 Recalage rigide sur $SO(2)$	21
Conclusion	25
Références	26
A Code Python	27
B Figures supplémentaires	28

Introduction

1 Le modèle de recalage LDDMM

Parmi les nombreuses approches de recalage, nous nous concentrerons sur le *Large Deformation Diffeomorphic Metric Mapping*¹. Sa force réside dans sa capacité à modéliser des déformations tout en garantissant que la transformation obtenue est un *difféomorphisme* - c'est-à-dire une application bijective, lisse et d'inverse lisse. Cette propriété est cruciale pour préserver la topologie des structures (anatomiques par exemple) : des ensembles connexes restent connexes, des surfaces lisses restent lisses, et il n'y a pas de nouvelles pliures ou de déchirures. Le cadre de *Large Deformation Diffeomorphic Metric Mapping* fournit une formulation variationnelle du problème de recalage, où la transformation recherchée est obtenue comme solution d'un problème de minimisation d'énergie.

L'objectif de ce chapitre est de présenter les fondements théoriques du modèle LDDMM.

1.1 Déformations et Difféomorphismes

En analyse de formes, on cherche à transformer une forme source I_0 vers une forme cible I_1 . Pour pouvoir effectuer cette transformation, on veut comparer ces deux formes en prenant en compte leurs propriétés géométriques. Et pour préserver la topologie de la forme, on ne déforme pas l'objet directement, mais l'espace ambiant via un flot de difféomorphisme $t \mapsto \varphi_t$ qui envoie la source sur la cible [5].

Définition 1.1 (Flot de difféomorphismes).

On considère une famille de difféomorphismes

$$\varphi_t : \mathbb{R}^d \rightarrow \mathbb{R}^d, \quad t \in [0, 1],$$

engendrée par un champ de vitesse dépendant du temps $v_t : \mathbb{R}^d \rightarrow \mathbb{R}^d$.

Le flot est défini comme la solution de l'équation différentielle :

$$\begin{cases} \dot{\varphi}_t = v_t \circ \varphi_t, \\ \varphi_0 = Id. \end{cases}$$

Cette équation décrit le transport des points de l'espace sous l'action du champ de vitesse v_t .

La déformation à l'instant t est donnée par l'action du difféomorphisme sur la forme : $I_1 = \varphi_t \cdot I_0$. Cette action dépend de la nature géométrique de la forme étudiée.

- **Points repères (Landmarks)** : Si $I_0 = (x_1, \dots, x_N) \in (\mathbb{R}^d)^N$, l'action est l'évaluation ponctuelle : $\varphi \cdot I_0 := \varphi(I_0)$.
- **Courbes paramétrées** : Si $q \in \mathcal{C}^1(I; \mathbb{R}^d)$, l'action est la composition à gauche : $\varphi \cdot q := \varphi \circ q$.

1. Que l'on appellera LDDMM par la suite

- **Images** : Si $I_0 \in \mathcal{L}^2(\Omega, \mathbb{R})$, l'action est la composition avec l'inverse (pour préserver l'intensité) : $\varphi \cdot I_0 := I_0 \circ \varphi^{-1}$.

Exemple 1.2. Voyons un exemple de recalage par rotation sur $SO(2)$ avec une forme source I_0 et une autre forme cible I_1 séparées par une rotation (cet exemple (5.2) sera détaillé plus loin). La forme source, à laquelle on a appliqué une rotation, recouvre parfaitement la forme cible :

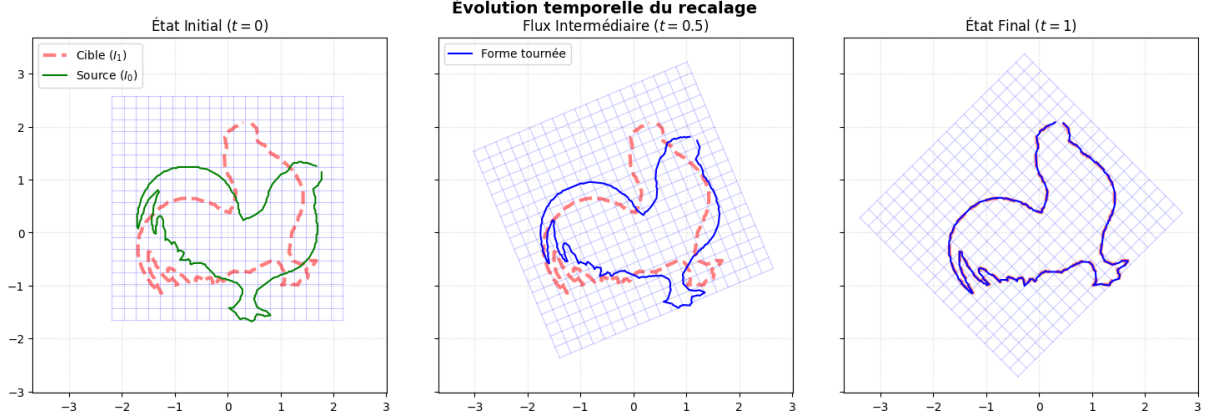


FIGURE 1.1 – Recalage sur les rotations $SO(2)$ d'un coq.

1.2 Problème variationnel LDDMM

Le problème de recalage consiste à déterminer le champ de vitesse v_t qui minimise une énergie composée de deux termes :

- Un terme de régularité contrôlant le “coût” de la déformation (à gauche de l'addition),
- Un terme d'attache aux données mesurant la similarité entre l'image transformée et la cible (à droite).

Le problème s'écrit :

$$\min_{v \in L^2([0,1], V)} J(v) = \frac{1}{2} \int_0^1 \|v_t\|_V^2 dt + \|I_0 \circ \varphi_1^{-1} - I_1\|_{L^2}$$

tel que $\begin{cases} \dot{\varphi}_t = v_t \circ \varphi_t \\ \varphi_0 = Id \end{cases}$

Proposition 1.3. Notre équation de minimisation admet bien un minimum.

Preuve. On utilise une méthode de calcul variationnel [4], “Direct method in the calculus of variations” pour montrer que l'équation de minimisation admet un minimum. $\square$

Remarque 1.4. Pour appliquer la méthode LDDMM, on n'a besoin que d'une action du groupe des difféomorphismes sur l'espace des données (ici les images). La formulation du problème de minimisation repose uniquement sur l'application du groupe à cet espace via $I_0 \mapsto I_0 \circ \varphi^{-1}$.

Ce chapitre a posé les fondements nécessaires à l'étude des cas particuliers, comme celui des rotations $SO(2)$ et $SO(3)$, qui seront abordés dans les chapitres suivants.

2 Le groupe $SO(n)$ et son algèbre de Lie $\mathfrak{so}(n)$

Dans ce chapitre, nous nous intéresserons aux particularités du groupe $SO(n)$ et de son algèbre de Lie $\mathfrak{so}(n)$ pour $n \geq 2$. Voici quelques définitions pour commencer.

Définition 2.1 (Variété différentielle).

Une **variété différentielle** est un espace qui est localement assimilable¹ à un espace euclidien [3].

Considérons G une variété différentielle.

Définition 2.2 (Espace tangent).

On appelle **espace tangent** en g à G , que l'on note $T_g G$, l'ensemble des vecteurs tangents en g .

2.1 Le groupe de Lie $SO(n)$

Le **groupe spécial orthogonal** $SO(n)$ est un sous-groupe du **groupe orthogonal** $O(n)$. C'est le groupe des matrices de rotation à n dimensions. On peut l'écrire :

$$SO(n) = \{A \in \mathcal{M}_n(\mathbb{R}) \mid \det A = 1 ; A^T A = I_n\}$$

Définition 2.3 (Groupe de Lie).

Un **groupe de Lie** est une variété différentielle qui est également un groupe, de telle sorte que l'opération de composition du groupe et l'opération d'inversion sont des applications lisses² [3].

Théorème 2.4 (Théorème de submersion).

Soient $f \in \mathcal{C}^1(\mathbb{R}^n, \mathbb{R}^m)$ et $G := f^{-1}(e)$ où $e \in \mathbb{R}^m$.

Si f est une submersion (i.e. si $Df(x)$ est surjective en tout point x de G), alors G est une sous-variété de l'espace $\mathbb{R}^n$ de dimension $n - m$.

Preuve. Admis □

Proposition 2.5. $SO(n)$ est un groupe de Lie

Preuve. Nous montrons d'abord que $SO(n)$ est un groupe (1), puis qu'il possède une structure de variété différentielle compatible avec les opérations de groupe (2).

1. On pourrait aussi dire : localement homéomorphe à $\mathbb{R}^n$

2. Lisse : de classe $\mathcal{C}^\infty$

(1) $SO(n)$ est un groupe pour le produit matriciel.

Soient $A, B \in SO(n)$, montrons que le produit est stable :

$$(AB)^T(AB) = B^T A^T AB = B^T B = I_n \quad \text{et} \quad \det(AB) = \det(A) \det(B) = 1.$$

Ainsi $AB \in SO(n)$.

L'associativité est celle du produit matriciel.

L'élément neutre est la matrice identité I_n , qui appartient à $SO(n)$ puisque

$$I_n^T I_n = I_n \quad \text{et} \quad \det(I_n) = 1.$$

Enfin, si $A \in SO(n)$, alors

$$A^T A = I_n,$$

d'où

$$A^{-1} = A^T.$$

Comme

$$\det(A^{-1}) = \frac{1}{\det(A)} = 1,$$

on obtient $A^{-1} \in SO(n)$.

Ainsi $SO(n)$ est un groupe et le produit matriciel est bien une loi de composition interne sur l'ensemble $SO(n)$.

(2) $SO(n)$ est une variété différentielle.

Montrons que $SO(n)$ est muni d'une structure de variété différentielle, comme sous-variété de l'espace $\mathcal{M}_n(\mathbb{R})$.

Considérons l'application

$$\begin{aligned} f: \mathcal{M}_n(\mathbb{R}) &\longrightarrow \mathcal{S}_n(\mathbb{R}) \\ X &\longmapsto X^T X \end{aligned}$$

où $\mathcal{S}_n(\mathbb{R})$ désigne l'espace des matrices symétriques.

On remarque que

$$O(n) = f^{-1}(I_n).$$

Calculons la différentielle de f .

Soient $X \in O(n)$, et $H \in \mathcal{M}_n(\mathbb{R})$:

$$\begin{aligned} f(X + H) &= (X + H)^T (X + H) \\ &= X^T X + X^T H + H^T X + H^T H \\ &= f(X) + X^T H + H^T X + H^T H \end{aligned}$$

Comme la norme subordonnée est sous-multiplicative

$$\|H^T H\| \leq \|H^T\| \|H\| = \|H\|^2,$$

on a

$$\begin{aligned} f(X + H) &= f(X) + X^T H + H^T X + O(\|H\|^2) \\ &= f(X) + Df(X)H + O(\|H\|^2) \end{aligned}$$

La différentielle de f en X est donc donnée par la partie linéaire en H ,

$$Df(X)H = X^T H + H^T X.$$

Il reste à montrer que f est une submersion en tout point de $O(n)$, donc que $Df(X)$ est surjective :

Soit $Y \in \mathcal{S}_n(\mathbb{R})$, déterminons H

$$Df(X)H = Y \Leftrightarrow X^T H + H^T X = Y.$$

Posons $H = \frac{1}{2}XY$ de telle sorte que pour $X \in O(n)$

$$\begin{aligned} Df(X)\frac{1}{2}XY = Y &\Leftrightarrow X^T \frac{1}{2}XY + (\frac{1}{2}XY)^T X = Y \\ &\Leftrightarrow \frac{1}{2}X^T XY + \frac{1}{2}Y^T X^T X = Y \end{aligned}$$

or $X^T X = I_n$ et $Y^T = Y$, donc

$$Df(X)\frac{1}{2}XY = \frac{1}{2}I_n Y + \frac{1}{2}Y I_n = Y.$$

On a trouvé pour n'importe quel Y , qu'il y a un antécédent H . Ainsi, Df est surjective et donc $O(n)$ est bien une sous-variété de l'espace $\mathcal{M}_n(\mathbb{R})$ (par le *Théorème de submersion 2.4*).

Posons $U = \{A \in \mathcal{M}_n(\mathbb{R}) \mid \det A > 0\}$ un ouvert (le déterminant est continu et U est l'image réciproque d'un ouvert par le déterminant).

L'intersection d'une sous-variété avec un ouvert de l'espace ambiant est encore une sous-variété. Comme $SO(n) = O(n) \cap U$, alors $SO(n)$ est une sous-variété. $\square$

Remarque 2.6. $SO(n)$ hérite aussi localement de toutes les propriétés de $O(n)$, comme sa dimension ou encore son espace tangent.

$$\begin{aligned} \dim(SO(n)) &= \dim(\text{Espace de départ}) - \dim(\text{Espace d'arrivée}) \\ &= n^2 - \frac{n(n+1)}{2} \\ &= \frac{n(n-1)}{2} \end{aligned}$$

2.1.1 Propriétés de l'action du groupe $SO(n)$ sur $\mathbb{R}^n$

Soient G un groupe et X un ensemble.

Définition 2.7 (Action de groupe).

On dit que G agit à gauche sur X s'il existe une application $u: G \times X \longrightarrow X$ vérifiant

- $u(e, x) = x$ pour tout $x \in X$ avec e l'élément neutre de G ,
- pour tout $g, g' \in G$ et $x \in X$

$$u(g, u(g', x)) = u(gg', x).$$

Remarque 2.8. On définit de même une action à droite par la propriété $A(g, A(g', x)) = A(g'g, x)$.

Proposition 2.9. *l'action naturelle du groupe $SO(n)$ sur $\mathbb{R}^n$ est donnée par la multiplication matricielle :*

$$\begin{aligned} u: SO(n) \times \mathbb{R}^n &\longrightarrow \mathbb{R}^n \\ (R, x) &\longmapsto Rx \end{aligned}$$

Preuve. Soient $R, R' \in SO(n)$ et $x \in \mathbb{R}^n$,

$$u(I_n, x) = I_n x = x.$$

Et

$$u(R, u(R', x)) = u(R, R'x) = RR'x = u(RR', x).$$

La multiplication matricielle u est une action de groupe. □

Proposition 2.10. *L'action du groupe $SO(n)$ dans $\mathbb{R}^n$ est fidèle mais pas libre et pas transitive.*

Preuve. L'action de $SO(n)$ dans $\mathbb{R}^n$ est **fidèle** :

Si l'action est fidèle, alors si $R \in SO(n)$ tel que $u(R, x) = x, \forall x \in \mathbb{R}^n$ alors $R = I_n$.

Supposons que $R \in SO(n)$ vérifie $Rx = x$ pour tout $x \in \mathbb{R}^n$. Alors, pour tout vecteur de la base canonique de $\mathbb{R}^n : (e_1, \dots, e_n)$, on a

$$Re_i = e_i \quad \forall 1 \leq i \leq n.$$

Ainsi, les colonnes de R sont celles de l'identité, donc $R = I_n$. L'action est fidèle.

L'action de $SO(n)$ dans $\mathbb{R}^n$ n'est **pas libre** :

Si l'action est libre, alors $\forall x \in \mathbb{R}^n$, si $x \neq 0$ et $R \neq I_n$ alors $Rx \neq x$.

Prenons $R = \begin{pmatrix} 1 & 0 \\ 0 & S \end{pmatrix}$ une matrice par bloc de $SO(n)$ où $S \in SO(n-1)$ différente de l'identité et $x = (1, 0, \dots, 0)$.

Alors $u(R, x) = Rx = x$ car x est dans la direction de la première colonne de R .

Donc l'action de $SO(n)$ dans $\mathbb{R}^n$ n'est pas libre.

L'action de $SO(n)$ dans $\mathbb{R}^n$ n'est **pas transitive** :

Si l'action est transitive, alors $\forall x, y \in \mathbb{R}^n, \exists R \in SO(n)$ tel que $u(R, x) = y$.

Prenons $x = 0$ et $y = (1, 0, \dots, 0)$.

Alors $\forall R \in SO(n), u(R, x) = Rx = R \cdot 0 = 0 \neq y$.

Donc l'action de $SO(n)$ dans $\mathbb{R}^n$ n'est pas transitive. □

Proposition 2.11. *L'action de $SO(n)$ sur $\mathbb{R}^n$ est isométrique. C'est-à-dire :*

$$\forall x, y \in \mathbb{R}^n, \forall R \in SO(n), \langle Rx, Ry \rangle = \langle x, y \rangle.$$

Preuve. Soient $x, y \in \mathbb{R}^n$ et $R \in SO(n)$

$$\langle Rx, Ry \rangle = \langle x, R^T Ry \rangle.$$

Or $R^T R = I_n$ car $R \in SO(n)$, donc

$$\begin{aligned} \langle Rx, Ry \rangle &= \langle x, I_n y \rangle \\ &= \langle x, y \rangle. \end{aligned}$$

L'action de $SO(n)$ sur $\mathbb{R}^n$ est isométrique, elle préserve bien la distance. □

Définition 2.12 (Orbite d'une action).

On appelle **orbite de x** la partie de l'ensemble X définie par

$$Orb(x) = \{u(g, x) \mid g \in G\}.$$

Remarque 2.13. Les orbites de l'action naturelle de $SO(n)$ sur $\mathbb{R}^n$ sont exactement les sphères centrées en l'origine (ainsi que le singleton $\{0\}$).

2.1.2 Propriétés topologiques de $SO(n)$

Théorème 2.14 (Compacité en dimension finie).

Soient $(X, \|\cdot\|)$ un espace vectoriel normé de dimension finie et $E \subset X$ une partie de X . Alors E est compacte si et seulement si E est fermée et bornée.

Preuve. $\Rightarrow$ Une partie compacte est aussi fermée et bornée. Cette implication est donc vraie dans n'importe quel espace et n'a rien à voir avec le cadre de la dimension finie!

$\Leftarrow$ C'est une conséquence du Théorème de Bolzano-Weierstrass. $\square$

Proposition 2.15. $SO(n)$ est un groupe compact.

Preuve. On a $SO(n) \subset \mathcal{M}_n(\mathbb{R})$ qui est en dimension finie.

Montrons que $SO(n)$ est fermé (1) et borné (2) :

(1) $SO(n)$ **fermé**.

Soient f et g deux applications continues.

$$\begin{aligned} f: \mathcal{M}_n(\mathbb{R}) &\longrightarrow \mathbb{R} & g: \mathcal{M}_n(\mathbb{R}) &\longrightarrow \mathcal{M}_n(\mathbb{R}) \\ X &\longmapsto \det(X) & X &\longmapsto X^T X \end{aligned}$$

On pose $\{1\} \subset \mathbb{R}$ et $\{Id\} \subset \mathcal{M}_n(\mathbb{R})$, deux fermés (car ce sont des singletons).

Alors $SO(n) = f^{-1}(\{1\}) \cap g^{-1}(\{Id\})$, comme l'intersection de deux fermés (images réciproques de fermés par des applications continues). Ainsi $SO(n)$ est fermé.

(2) $SO(n)$ **borné**.

Soit $A \in SO(n)$, alors $\|A\|_F^2 = \text{Tr}(A^T A) = \text{Tr}(Id) = n$.

On a donc : $\forall A \in SO(n), \exists M = \sqrt{n}$ tel que $\|A\| \leq M$ (toutes les normes sont équivalentes en dimension finie).

On retrouve que $SO(n)$ est borné car $SO(n) \subset B(0, M)$.

Finalement, $SO(n)$ fermé et borné donc (par le *Théorème de compacité en dimension finie 2.14*) $SO(n)$ est compact. $\square$

Proposition 2.16. $SO(n)$ est un groupe connexe par arcs.

Preuve. On procède par récurrence sur $n \geq 1$ en utilisant l'action naturelle de $SO(n)$ sur la sphère $S^{n-1} \subset \mathbb{R}^n$.

$$\begin{aligned} u: SO(n) \times S^{n-1} &\longrightarrow S^{n-1} \\ (A, x) &\longmapsto Ax \end{aligned}$$

Pour $n = 1$, le groupe $SO(1) = \{1\}$ est un singleton, il est donc trivialement connexe par arcs.

Vérifions que le fixateur du vecteur $e_n = (0, \dots, 0, 1)^T$ s'identifie à $SO(n-1)$.

Une matrice $A \in SO(n)$ vérifie $Ae_n = e_n$ si et seulement si sa dernière colonne est e_n . Étant orthogonale, ses lignes et colonnes forment des familles orthonormées, ce qui impose à A d'avoir une structure de bloc de la forme :

$$A = \begin{pmatrix} B & 0 \\ 0 & 1 \end{pmatrix}$$

où $B \in M_{n-1}(\mathbb{R})$. Les conditions $A^T A = I_n$ et $\det(A) = 1$ sont aussi vérifiées sur le bloc B .

Donc, $B \in SO(n-1)$. L'application $B \mapsto \begin{pmatrix} B & 0 \\ 0 & 1 \end{pmatrix}$ définit un isomorphisme qui permet d'identifier le fixateur de e_n à $SO(n-1)$.

Montrons que l'application $SO(n)/SO(n-1) \longrightarrow S^{n-1}$ ainsi obtenue est un homéomorphisme.

L'action de $SO(n)$ sur S^{n-1} est transitive pour tout $n \geq 2$ (tout vecteur unitaire peut être complété en une base orthonormée). L'application associée à e_n est donc surjective :

$$\begin{aligned} \varphi: SO(n) &\longrightarrow S^{n-1} \\ A &\longmapsto Ae_n \end{aligned}$$

Par passage au quotient, φ induit une bijection continue :

$$\varphi': SO(n)/SO(n-1) \longrightarrow S^{n-1}$$

Le groupe $SO(n)$ étant compact (2.15), toute bijection continue d'un espace compact vers un espace séparé est un homéomorphisme.

On en déduit que $SO(n)/SO(n-1) \simeq S^{n-1}$.

Montrons que si H est un sous-groupe fermé connexe de G , l'application quotient induit une bijection $\pi_0(G) \simeq \pi_0(G/H)$.

En effet, soit $f: G \rightarrow \{0, 1\}$ une application continue. Pour tout $g \in G$, la restriction de f à la classe gH est continue. Comme H est connexe, la fibre³ gH est connexe, donc f est constante. Donc, f passe au quotient en une application continue $\bar{f}$. Si G/H est connexe, $\bar{f}$ est constante, impliquant que f est constante sur G .

G est donc connexe.

On en déduit la connexité de $SO(n)$ par récurrence.

Supposons que $SO(n-1)$ est connexe. On applique l'égalité précédente à $G = SO(n)$ et $H = SO(n-1)$.

Par hypothèse, H est connexe. De plus, pour $n \geq 2$, l'espace quotient $SO(n)/SO(n-1) \simeq S^{n-1}$ est connexe.

$SO(n)$ est donc connexe.

Et $SO(n)$ est connexe par arcs car c'est une variété différentielle (2.5) localement connexe par arcs et compacte (2.15). $\square$

2.2 L'algèbre de Lie $\mathfrak{so}(n)$

L'outil principal qui nous permettra d'étudier les groupes de Lie est leur algèbre de Lie. Définissons cette structure pour les rotations et intéressons-nous aux propriétés différentielles des groupes de Lie. Prenons G un groupe de Lie et $e \in G$ son élément neutre.

Définition 2.17 (Algèbre de Lie).

Une **algèbre de Lie**, notée $\mathfrak{g} = T_e G$, est un espace vectoriel muni d'un produit appelé commutateur (ou crochet de Lie), qui est une application bilinéaire antisymétrique [3]

$$[\cdot, \cdot] : \mathfrak{g} \times \mathfrak{g} \rightarrow \mathfrak{g}$$

3. En topologie, la fibre d'un point par une application désigne simplement sa préimage

et qui satisfait l'identité de Jacobi :

$$[\xi, [\eta, \zeta]] + [\eta, [\zeta, \xi]] + [\zeta, [\xi, \eta]] = 0, \quad \text{pour } (\xi, \eta, \zeta) \in \mathfrak{g}.$$

Définition 2.18 (Crochet de Lie dans $\mathfrak{so}(n)$).

Le commutateur de $\mathfrak{so}(n)$ est, pour $A, B \in \mathfrak{so}(n)$:

$$\begin{aligned} [\cdot, \cdot] : \mathfrak{g} \times \mathfrak{g} &\rightarrow \mathfrak{g} \\ (A, B) &\mapsto AB - BA \end{aligned}$$

Proposition 2.19. *Les algèbres de Lie des groupes $SO(n)$ et $O(n)$ coïncident :*

$$\mathfrak{so}(n) = \mathfrak{o}(n)$$

Preuve. Montrons par double inclusion :

$\mathfrak{so}(n) \subset \mathfrak{o}(n)$: Comme $SO(n) \subset O(n)$, tout vecteur tangent à $SO(n)$ en l'identité est aussi tangent à $O(n)$.

$$T_{Id}SO(n) \subset T_{Id}O(n), \text{ donc } \mathfrak{so}(n) \subset \mathfrak{o}(n).$$

$\mathfrak{so}(n) \supset \mathfrak{o}(n)$: On utilise le fait que $SO(n)$ est la composante connexe de l'identité dans $O(n)$ (2.16).

Au voisinage de Id , les deux groupes coïncident. En effet, dans un voisinage de Id , le déterminant reste positif car il vaut 1 en Id et la fonction $\det$ est continue.

Donc localement, toute courbe dans $O(n)$ issue de Id reste dans $SO(n)$.

Ainsi, tous les espaces tangents à $O(n)$ en Id le sont aussi pour $SO(n)$, donc :

$$T_{Id}O(n) \subset T_{Id}SO(n)$$

Finalement, on a $\mathfrak{so}(n) = \mathfrak{o}(n)$. □

Proposition 2.20. $T_{Id}SO(n) = \{H \in \mathcal{M}_n(\mathbb{R}) \mid H^T = -H\}$ est l'algèbre de Lie de $SO(n)$. Plus précisément : $\mathfrak{so}(n) = T_{Id}SO(n)$.

Preuve. $SO(n)$ est un groupe de Lie (2.5). L'algèbre de Lie d'un groupe de Lie est définie comme son espace tangent en l'identité (ici Id), que l'on trouve en différenciant f^4 en l'identité avec

$$\begin{aligned} f : \mathcal{M}_n(\mathbb{R}) &\longrightarrow \mathcal{S}_n(\mathbb{R}) \\ X &\longmapsto X^T X \end{aligned}$$

Par la preuve de la proposition (2.5), on a

$$Df(X)H = X^T H + H^T X.$$

On l'évalue en l'identité

$$Df(Id)H = I_n^T H + H^T I_n = H + H^T.$$

Enfin,

$$T_{Id}SO(n) = \ker(Df(Id)) = \{H \in \mathcal{M}_n(\mathbb{R}) \mid H^T = -H\}$$

Par définition, l'algèbre de Lie de $SO(n)$ est l'espace tangent en l'identité.

Ainsi, l'algèbre de Lie du groupe des rotations de dimension n est l'ensemble des matrices antisymétriques.

$$\mathfrak{so}(n) = \{H \in \mathcal{M}_n(\mathbb{R}) \mid H^T = -H\}$$

□

4. En effet, on peut ignorer la condition sur le déterminant car on a déjà montré que les algèbres de Lie de $SO(n)$ et $O(n)$ coïncident (2.19)

Proposition 2.21. Si $A, B \in \mathfrak{so}(n)$, Alors $[A, B] \in \mathfrak{so}(n)$

Preuve. Soient $A, B \in \mathfrak{so}(n)$, calculons la transposée de $[A, B]$

$$\begin{aligned} [A, B]^T &= (AB - BA)^T \\ &= (AB)^T - (BA)^T \\ &= B^T A^T - A^T B^T. \end{aligned}$$

On utilise l'antisymétrie de A et B

$$\begin{aligned} [A, B]^T &= (-B)(-A) - (-A)(-B) \\ &= -(AB - BA) = -[A, B] \end{aligned}$$

On a bien $[A, B]$ antisymétrique, donc $[A, B] \in \mathfrak{so}(n)$. □

Définition 2.22 (Translation à droite).

Soit $g \in G$. On note **translation à droite** l'application :

$$\begin{aligned} R_g : G &\longrightarrow G \\ h &\longmapsto hg \end{aligned}$$

Proposition 2.23. La translation à droite sur le groupe $SO(n)$ est une application lisse.

Preuve. Soit R_M une translation à droite sur $SO(n)$. On peut définir cette translation comme :

$$\begin{aligned} R_M : SO(n) &\longrightarrow SO(n) \\ X &\longmapsto u \times v = XM \end{aligned}$$

avec u l'application identité et v une application constante sur $SO(n)$. Par produit d'applications lisses (avec le produit matriciel), on a que R_M est une application lisse sur $SO(n)$. □

Proposition 2.24. L'espace $T_A SO(n)$ est isomorphe à $T_{Id} SO(n)$ pour tout $A \in SO(n)$

Preuve. Soit $A \in SO(n)$. On considère la translation à droite (2.22) sur $SO(n)$ définie par

$$\begin{aligned} R_A : SO(n) &\longrightarrow SO(n) \\ X &\longmapsto XA \end{aligned}$$

Cette application est lisse (2.23), et son inverse est $R_{A^{-1}}$. Ainsi, R_A est un difféomorphisme de $SO(n)$.

En particulier, sa différentielle en tout point est un isomorphisme linéaire. En évaluant en l'identité, on obtient

$$T_{Id} R_A : T_{Id} SO(n) \longrightarrow T_A SO(n),$$

qui est un isomorphisme. Son inverse est donné par

$$T_A R_{A^{-1}} : T_A SO(n) \longrightarrow T_{Id} SO(n),$$

et par théorème de dérivation des fonctions composées, on a

$$T_A R_{A^{-1}} \circ T_{Id} R_A = T_{Id} (R_{A^{-1}} \circ R_A) = \text{Id}_{T_{Id} SO(n)}.$$

On en déduit que $T_A SO(n)$ et $T_{Id} SO(n)$ sont isomorphes. □

Remarque 2.25. La translation à droite permet de “passer”⁵ d'un espace tangent à un autre, reliant l'espace tangent en un point quelconque et l'espace tangent à l'identité (2.24).

5. Passer : transporter de manière isomorphe les structures

2.3 L'application exponentielle

Définition 2.26 (Exponentielle de matrice).

Soit $A \in \mathcal{M}_n(\mathbb{K})$ avec $\mathbb{K} = \mathbb{R}$ ou $\mathbb{C}$. On appelle exponentielle de la matrice A , notée e^A , la somme de la série :

$$e^A = \sum_{k=0}^{\infty} \frac{A^k}{k!}, \quad A^0 = I_n, \quad \exp(0) = I_n.$$

Proposition 2.27. Si $A \in \mathfrak{so}(n)$, Alors $e^A \in SO(n)$.

Preuve. Montrons que $\det(e^A) = 1$ et $(e^A)^T e^A = I_n$, et donc que $e^A \in SO(n)$.

Soit $A \in \mathfrak{so}(n)$, A est une matrice antisymétrique.

On sait que pour tout $M \in \mathcal{M}_n(\mathbb{R})$

$$\det(e^M) = e^{\text{Tr}(M)}.$$

Par antisymétrie de A , $\text{Tr}(A) = 0$.

Et donc

$$\det(e^A) = e^{\text{Tr}(A)} = e^0 = 1.$$

Maintenant, montrons que $(e^A)^T e^A = I_n$.

Pour la même matrice A ,

$$\begin{aligned} (e^A)^T &= \left(\sum_{n=0}^{\infty} \frac{A^n}{n!} \right)^T = \sum_{n=0}^{\infty} \frac{(A^n)^T}{n!} \\ &= \sum_{n=0}^{\infty} \frac{(A^T)^n}{n!} = e^{(A^T)}. \end{aligned}$$

Or $A^T = -A$,

$$(e^A)^T (e^A) = (e^{-A})(e^A) = e^0 = I_n.$$

Ainsi, e^A vérifie $\det(e^A) = 1$ et $(e^A)^T e^A = I_n$.

On a bien $e^A \in SO(n)$. □

Exemple 2.28. Exemple d'exponentielle d'une matrice 2×2 antisymétrique :

Soient $\theta \in \mathbb{R}$ et $A = \begin{pmatrix} 0 & -\theta \\ \theta & 0 \end{pmatrix} \in \mathfrak{so}(2)$. Calculons e^A :

On observe que

$$A^2 = \begin{pmatrix} -\theta^2 & 0 \\ 0 & -\theta^2 \end{pmatrix} = -\theta^2 I_2,$$

et par récurrence,

$$A^{2k} = (-1)^k \theta^{2k} I_2, \quad A^{2k+1} = (-1)^k \theta^{2k+1} \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}.$$

Alors

$$\begin{aligned} e^A &= \sum_{k=0}^{\infty} \frac{A^k}{k!} = \sum_{k=0}^{\infty} \frac{A^{2k}}{(2k)!} + \sum_{k=0}^{\infty} \frac{A^{2k+1}}{(2k+1)!} \\ &= \left(\sum_{k=0}^{\infty} \frac{(-1)^k \theta^{2k}}{(2k)!} \right) I_2 + \left(\sum_{k=0}^{\infty} \frac{(-1)^k \theta^{2k+1}}{(2k+1)!} \right) \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}. \end{aligned}$$

On reconnaît les séries du cosinus et du sinus :

$$\cos \theta = \sum_{k=0}^{\infty} \frac{(-1)^k \theta^{2k}}{(2k)!}, \quad \sin \theta = \sum_{k=0}^{\infty} \frac{(-1)^k \theta^{2k+1}}{(2k+1)!}.$$

Ainsi,

$$e^A = \cos \theta I_2 + \sin \theta \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix},$$

qui est bien une matrice de rotation, donc $e^A \in SO(2)$.

Proposition 2.29. *L'application exponentielle $\exp: \mathfrak{so}(n) \longrightarrow SO(n)$ est surjective.*

Preuve. Soit $R \in SO(n)$. Par le théorème spectral, il existe une base orthonormée dans laquelle la matrice de R est diagonale par blocs, les blocs diagonaux étant soit des blocs de rotation :

$$R(\theta_k) = \begin{pmatrix} \cos \theta_k & -\sin \theta_k \\ \sin \theta_k & \cos \theta_k \end{pmatrix},$$

soit des blocs (1) et (-1) .

Comme $\det R = 1$, les valeurs propres (-1) apparaissent en nombre pair ; on les regroupe donc deux à deux pour former des blocs

$$-I_2 = R(\pi).$$

On obtient ainsi une décomposition

$$R = P \operatorname{diag}(R(\theta_1), \dots, R(\theta_r), 1, \dots, 1) P^{-1},$$

où P est orthogonale.

Pour chaque bloc de rotation d'angle θ_k , on construit explicitement

$$A_k = \begin{pmatrix} 0 & -\theta_k \\ \theta_k & 0 \end{pmatrix} \in \mathfrak{so}(2),$$

de sorte que

$$e^{A_k} = R(\theta_k).$$

On pose alors

$$A = P \operatorname{diag}(A_1, \dots, A_r, 0, \dots, 0) P^{-1}.$$

Comme $\mathfrak{so}(n)$ est stable par conjugaison orthogonale, on a $A \in \mathfrak{so}(n)$.

Finalement,

$$e^A = P \operatorname{diag}(e^{A_1}, \dots, e^{A_r}, 1, \dots, 1) P^{-1} = R.$$

Ainsi, on retrouve que $e^A = R$, et donc que pour tout $R \in SO(n)$, il existe $A \in \mathfrak{so}(n)$ tel que $e^A = R$. L'exponentielle est surjective de l'algèbre de Lie de $SO(n)$ dans son groupe de Lie. $\square$

3 Étude de l'équation différentielle linéaire $\dot{x}(t) = Ax(t)$

Dans ce chapitre, nous verrons comment cette équation différentielle linéaire apparaît naturellement sur le groupe spécial orthogonal $SO(n)$, l'importance de ce résultat, ainsi que sa résolution. Commençons avec le groupe de Lie $SO(n)$.

Exemple 3.1. *La translation à droite sur le groupe de Lie $SO(n)$ permet, comme on l'a vu, de "passer" d'un espace tangent à un autre de manière isomorphe.*

*La figure ci-dessous **Figure (??)** illustre ce principe sur la surface $z = x^2 - y^2$ (qui joue ici le rôle d'une variété différentielle). Le long de la courbe $y = 1$, on a représenté les plans tangents $T_{g_0}G$ et $T_{g_t}G$ en $g_0 = (-2, 1, 3)$ et $g_t = (0, 1, -1)$, ainsi que les vecteurs tangents correspondants. Le passage de l'un à l'autre peut être réalisé par une translation à droite convenable.*

3.1 Obtenir l'équation différentielle $\dot{x}(t) = Ax(t)$

Cherchons à exprimer la différentielle de la translation à droite afin de lier le groupe $SO(n)$ à son algèbre de Lie. Remarquons une égalité que l'on utilisera juste après.

Remarque 3.2. On sait que $\forall A \in SO(n): AA^{-1} = I_n$.

Trouvons l'équation.

Prenons la translation à droite sur $SO(n)$ avec $M \in SO(n)$:

$$\begin{aligned} R_M: SO(n) &\longrightarrow SO(n) \\ X &\longmapsto XM \end{aligned}$$

Différencions cette application, soient $X \in SO(n)$ et $A \in \mathcal{M}_n(\mathbb{R})$

$$\begin{aligned} R_M(X + A) &= (X + A)M \\ &= XM + AM \\ &= R_M(X) + DR_M(X)A + o(\|A\|) \end{aligned}$$

La différentielle de R_M en X est donc donnée par la partie linéaire en A ,

$$\begin{aligned} DR_M(A): T_A SO(n) &\longrightarrow T_{AM} SO(n) \\ X &\longmapsto AM \end{aligned}$$

Car

$$DR_M: SO(n) \longrightarrow \mathcal{L}(T_A SO(n), T_{AM} SO(n))$$

est à valeurs dans les applications linéaires d'un espace tangent dans un autre, qui sont isométriques (2.24).

Considérons maintenant une courbe différentiable $x(t) \in SO(n)$,

$$x(t) = M_t x_0$$

que l'on différencie par rapport au temps,

$$\dot{x}(t) = \dot{M}_t x_0.$$

Or

$$\dot{M}_t = A(t) M_t.$$

Et en posant $A(t) = \dot{x}(t)x(t)^{-1}$, on remarque (3.2) que $A(t) \in T_{I_n}SO(n) = \mathfrak{so}(n)$ (l'algèbre de Lie des matrices antisymétriques).

Donc si on suppose que ce vecteur est constant en fonction du temps $A(t) = A$,

$$\dot{x}(t) = A M_t x_0$$

$$\dot{x}(t) = A x(t)$$

Remarque 3.3. La solution de l'équation différentielle linéaire ci-dessus permet, à partir d'un seul élément A de l'algèbre de Lie $\mathfrak{so}(n)$, de créer un mouvement sur tout le groupe $SO(n)$ à partir d'applications de translations à droite.

3.2 Solution de $\dot{x}(t) = Ax(t)$

Pour garantir l'existence et l'unicité des trajectoires sur le groupe, revenons au cas général avec le théorème suivant.

Théorème 3.4 (Théorème de Cauchy-Lipschitz).

Soient Ω un ouvert de $\mathbb{R} \times \mathbb{R}^d$ et $F: \Omega \longrightarrow \mathbb{R}^d$. Supposons que :

- F est continue sur Ω .
- F est localement Lipschitzienne par rapport à x .

Alors, pour tout $(t_0, x_0) \in \Omega$, le problème de Cauchy :

$$(E) \begin{cases} Z'(t) = F(t, Z(t)) \\ Z(t_0) = x_0 \end{cases}$$

admet une unique solution maximale (J, x) .

Preuve. Admis □

Proposition 3.5. La solution de l'équation différentielle linéaire $\dot{x}(t) = Ax(t)$ est :

$$x(t) = e^{tA} x_0$$

Preuve. Dans notre cas, en associant $\mathbb{R}^d$ à $\mathcal{M}_n(\mathbb{R})$, l'application $F(t, X) = AX$ est linéaire par rapport à X , donc globalement lipschitzienne.

Le *Théorème de Cauchy-Lipschitz 3.4* s'applique et assure l'existence d'une unique solution maximale définie sur $\mathbb{R}$ tout entier, dont la forme explicite fait intervenir l'exponentielle de matrice.

Vérifions que notre unique solution est $x(t) = e^{tA}x_0$:
 x étant lisse, on peut écrire :

$$\begin{aligned}x(t) &= e^{tA}x_0 \\ \dot{x}(t) &= (\dot{e^{tA}})x_0\end{aligned}$$

On dérive en fonction de t .

$$\begin{aligned}&= Ae^{tA}x_0 \\ \dot{x}(t) &= Ax(t)\end{aligned}$$

Avec

$$x(0) = I_n x_0 = x_0$$

Ainsi, l'unique solution de l'équation différentielle est $x(t) = e^{tA}x_0$.

□

4 Cas particuliers de $SO(2)$ et $SO(3)$

Dans ce chapitre, nous sortirons un peu de la théorie en nous intéressant à des cas simples : les groupes de rotations $SO(2)$ et $SO(3)$.

Ces deux groupes vont nous donner un aperçu plus concret du cadre $LDDMM$.

4.1 $SO(2)$, Les rotations en dimension 2

Le groupe $SO(2)$ est très simple, il décrit les rotations du plan autour de l'origine en dimension 2.

Proposition 4.1. *Soit R une matrice de $SO(2)$, alors il existe $\theta \in \mathbb{R}$ tel que*

$$R = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$$

Preuve. Soit

$$R = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \in SO(2).$$

Comme $R^T R = I_2$, les colonnes de R forment une base orthonormée de $\mathbb{R}^2$. On a donc

$$a^2 + c^2 = 1.$$

Il existe alors $\theta \in \mathbb{R}$ tel que

$$a = \cos \theta, \quad c = \sin \theta.$$

La seconde colonne doit être un vecteur unitaire orthogonal à $(a, c)^T$. Elle est égale à

$$(-\sin \theta, \cos \theta)^T,$$

ou à

$$(\sin \theta, -\cos \theta)^T.$$

Or $\det(R) = 1$, ce qui élimine la seconde possibilité. Ainsi

$$b = -\sin \theta, \quad d = \cos \theta.$$

Ainsi,

$$R = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}.$$

□

Études l'équation $\dot{R}(t) = AR(t)$ dans $SO(2)$.

L'inconnue est une rotation que l'on va nommer ici R . On remarque que $A \in \mathfrak{so}(2)$ pour que la solution appartienne à $SO(2)$.

Alors on peut écrire A sous la forme

$$A = \begin{pmatrix} 0 & -\omega \\ \omega & 0 \end{pmatrix}, \text{ avec } \omega \in \mathbb{R}.$$

On a vu que la solution d'une équation différentielle linéaire de ce type est $e^{tA}x_0$ (3.5). Dans notre cas, on a donc

$$R(t) = e^{tA}R_0.$$

Or l'exponentielle d'une matrice de $\mathfrak{so}(n)$ est à valeur dans $SO(n)$ (2.27).

Sachant que $tA \in \mathfrak{so}(2)$, donc $e^{tA} \in SO(2)$ et e^{tA} peut s'écrire (4.1) comme

$$e^{tA} = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}, \text{ avec } \theta \in \mathbb{R} \text{ dépendant de } t.$$

Après résolution, on trouve

$$e^{tA} = \begin{pmatrix} \cos(\omega t) & -\sin(\omega t) \\ \sin(\omega t) & \cos(\omega t) \end{pmatrix}.$$

Par conséquent,

$$R(t) = \begin{pmatrix} \cos(\omega t) & -\sin(\omega t) \\ \sin(\omega t) & \cos(\omega t) \end{pmatrix} R_0.$$

Comme toute matrice de $SO(2)$ est une rotation, il existe $\theta_0 \in \mathbb{R}$ tel que

$$R_0 = \begin{pmatrix} \cos \theta_0 & -\sin \theta_0 \\ \sin \theta_0 & \cos \theta_0 \end{pmatrix}.$$

On obtient alors

$$R(t) = \begin{pmatrix} \cos(\omega t) & -\sin(\omega t) \\ \sin(\omega t) & \cos(\omega t) \end{pmatrix} \begin{pmatrix} \cos(\theta_0) & -\sin(\theta_0) \\ \sin(\theta_0) & \cos(\theta_0) \end{pmatrix}.$$

En effectuant le produit matriciel et en utilisant les formules d'addition du sinus et du cosinus, on trouve

$$R(t) = \begin{pmatrix} \cos(\theta_0 + \omega t) & -\sin(\theta_0 + \omega t) \\ \sin(\theta_0 + \omega t) & \cos(\theta_0 + \omega t) \end{pmatrix}. (*)$$

Ainsi, la solution est une rotation dont l'angle évolue linéairement avec le temps.

Application au modèle de recalage.

Soit x_S notre forme source et x_T notre forme cible. Rappelons notre équation de minimisation dans le cas général [6] pour I_0 la forme source et I_1 la forme cible :

$$\begin{aligned} \min_{v \in L^2([0,1], V)} J(v) &= \frac{1}{2} \int_0^1 \|v_t\|_V^2 dt + \|I_0 \circ \varphi_1^{-1} - I_1\| \\ \text{tel que } \begin{cases} \dot{\varphi}_t &= v_t \circ \varphi_t \\ \varphi_0 &= Id \end{cases} \end{aligned}$$

Dans $SO(2)$, l'espace des transformations est réduit aux rotations du plan. On cherche la meilleure rotation, que l'on peut noter x_1 , vérifiant

$$\begin{cases} \dot{x}(t) = A_t x(t) \\ x_0 = x_S \end{cases}$$

Alors, on résoud l'équation différentielle ci-dessus en utilisant la solution (*)

$$x(t) = R(t)x_S.$$

On obtient ainsi une description explicite de l'évolution de la forme source sous l'action d'une rotation. Le résultat x_1 apparaît quand la rotation est totalement appliquée à notre forme source.

Maintenant, regardons l'énergie cinétique dans le cas des rotations de dimension 2. Avec le produit matricielle en loi de composition, on retrouve naturellement

$$v_t = A_t, \quad A \in \mathfrak{so}(n).$$

Ainsi, notre minimisation ressemble maintenant à ça (où $Skew_2$ désigne l'espace des matrices antisymétriques.) :

$$\begin{aligned} \min_{A \in L^2([0,1], Skew_2)} \quad & \frac{1}{2} \int_0^1 \|A_t\|_{Skew_2}^2 dt + \|x_1 - x_T\| \\ \text{tel que} \quad & \begin{cases} \dot{x}(t) = A_t x(t) \\ x_0 = x_S \end{cases} \end{aligned}$$

Or A étant antisymétrique, donc sous la forme $A_t = \begin{pmatrix} 0 & -\omega_t \\ \omega_t & 0 \end{pmatrix}$ pour un certain $\omega_t \in \mathbb{R}$.

On peut simplifier notre minimisation, et utiliser la valeur absolue au lieu d'une norme qui convient :

$$\begin{aligned} \min_{\omega \in \mathbb{R}} \quad & \frac{1}{2} \int_0^1 |\omega_t|^2 dt + \|x_1 - x_T\| \\ \text{tel que} \quad & \begin{cases} \dot{x}(t) = A_t x(t) \\ x_0 = x_S \end{cases} \end{aligned}$$

Si l'on cherche une géodésique¹, on pose $\omega_t = \omega$ pour tout t . Le problème se réduit à l'optimisation d'un seul scalaire paramétrique $\omega \in \mathbb{R}$:

$$\begin{aligned} \min_{\omega \in \mathbb{R}} \quad & J(\omega) = \frac{1}{2} |\omega|^2 + \|x_1(\omega) - x_T\| \\ \text{tel que} \quad & \begin{cases} \dot{x}(t) = A_t x(t) \\ x_0 = x_S \end{cases} \end{aligned}$$

Autrement dit, dans le cas de $SO(2)$, le problème de recalage consiste à rechercher une vitesse d'angle² optimale permettant de faire coïncider au mieux la forme source avec la forme cible tout en "optimisant"³ la rotation.

1. La rotation la plus naturelle, sans accélération, à vitesse angulaire constante
2. Vitesse d'angle ou vitesse angulaire : un angle qui dépend du temps ω_t
3. Optimiser : minimiser le coût énergétique

5 Implémentation algorithmique

Nous détaillons ici l'algorithme de résolution numérique pour les rotations, l'implémentation du code en Python, ainsi que l'analyse des résultats expérimentaux obtenus sur différentes formes tests.

5.1 Algorithme de résolution - Discrétiser puis Optimiser

Pour résoudre ce problème numériquement, on utilise un algorithme itératif :

— **Initialisation** : On choisit une vitesse angulaire de départ, par exemple

$$\omega^{(0)} = 0.$$

— **Boucle d'optimisation** (pour chaque itération k) :

1. *Intégration du flot* (méthode d'Euler explicite) : On discrétise le temps physique $t \in [0, 1]$ en N sous-intervalles de pas Δt . En partant de la position initiale $x_{t=0} = x_S$, on intègre la dynamique pas à pas :

$$x_{t+\Delta t} = x_t + \Delta t A(\omega^{(k)}) x_t.$$

On obtient ainsi la position déformée finale x_{finale} , correspondant à l'instant $t = 1$.

2. *Évaluation de l'énergie* : On calcule le coût global

$$J(\omega^{(k)}).$$

3. *Mise à jour* (descente de gradient) : On calcule le gradient de l'énergie $\nabla J(\omega^{(k)})$ et on actualise la vitesse angulaire avec un taux d'apprentissage α :

$$\omega^{(k+1)} = \omega^{(k)} - \alpha \nabla J(\omega^{(k)}).$$

La descente de gradient peut être réalisée à l'aide de l'algorithme L-BFGS de PyTorch.

5.1.1 Recalage rigide sur $SO(2)$

On souhaite toujours minimiser le problème suivant :

$$\begin{aligned} \min_{v \in L^2([0,1], V)} J(v) &= \frac{1}{2} \int_0^1 \|v_t\|_V^2 dt + \|I_0 \circ \varphi_1^{-1} - I_1\| \\ \text{tel que } &\begin{cases} \dot{\varphi}_t = v_t \circ \varphi_t \\ \varphi_0 = Id \end{cases} \end{aligned}$$

On remplace le champ de vitesse LDDMM par un champ rigide

$$\dot{x} = A(\omega)x,$$

avec

$$A(\omega) = \begin{pmatrix} 0 & -\omega \\ \omega & 0 \end{pmatrix} \in \mathfrak{so}(2).$$

Le paramètre $\omega \in \mathbb{R}$ est la vitesse angulaire.

On retrouve alors le problème présenté dans le chapitre 4 **Cas particuliers de $SO(2)$ et $SO(3)$** , avec x_S notre forme source et x_T notre forme cible :

$$\begin{aligned} \min_{\omega \in \mathbb{R}} J(\omega) &= \frac{1}{2}|\omega|^2 + \|x_1(\omega) - x_T\| \\ \text{tel que } &\begin{cases} \dot{x} = A(\omega)x \\ x_{t=0} = x_S \end{cases} \end{aligned}$$

Méthode pour le recalage rigide sur $SO(2)$.

Étape 1 : *Intégration du flot*

La fonction `Euler` (A.1) intègre le flot rigide par la méthode d'Euler explicite sur $t \in [0, 1]$.

Étape 2 : *Évaluation de l'énergie*

La fonction `rotation_loss` (A.2) calcule l'énergie totale $J(\omega) = \frac{1}{2}|\omega|^2 + \|x_1(\omega) - x_T\|$.

Étape 3 : *Mise à jour (descente de gradient)*

La fonction `opti_rotation` (A.3) optimisation de la vitesse angulaire ω par descente de gradient.

Exemple 5.1 (Application de la méthode et calcul de la première itération).

On initialise le paramètre de rotation à :

$$\omega^{(0)} = 0$$

Cette valeur correspond à une rotation nulle. On évalue alors la fonction coût associée au recalage :

$$J(\omega^{(0)}) = J(0) = 9.488581$$

*Cette valeur mesure l'écart entre la forme de référence et la forme transformée lorsque aucune rotation n'est appliquée. La valeur ici provient de l'exemple **Figure (??)**.*

Calcul du gradient

Le gradient de la fonction coût en $\omega^{(0)}$ est :

$$\nabla J(\omega^{(0)}) = 23.035329$$

Ce gradient est positif, ce qui signifie que la fonction coût augmente lorsque ω augmente localement. Pour diminuer la fonction coût, il faut donc se déplacer dans la direction opposée au gradient.

Application de la descente de gradient

La formule de mise à jour est :

$$\omega^{(k+1)} = \omega^{(k)} - \alpha \nabla J(\omega^{(k)})$$

avec :

$$\alpha = 10^{-3}$$

En remplaçant par les valeurs numériques :

$$\omega^{(1)} = 0 - 10^{-3} \times 23.035329$$

On calcule d'abord le déplacement

$$10^{-3} \times 23.035329 = 0.023035329$$

Puis

$$\omega^{(1)} = 0 - 0.023035329$$

d'où

$$\boxed{\omega^{(1)} = -0.023035329}$$

Vérification de l'amélioration

Après cette mise à jour, on réévalue la fonction coût :

$$J(\omega^{(1)}) = 8.950083$$

Comme

$$8.950083 < 9.488581,$$

la fonction coût a diminué. La première étape de la descente de gradient améliore donc le recalage en rapprochant les deux formes.

Résumé de l'itération :

$$\omega^{(0)} = 0$$

$$\nabla J(\omega^{(0)}) = 23.035329$$

$$\Delta\omega = -\eta\nabla J(\omega^{(0)}) = -0.023035329$$

$$\omega^{(1)} = -0.023035329$$

$$J(\omega^{(0)}) = 9.488581 \quad \rightarrow \quad J(\omega^{(1)}) = 8.950083$$

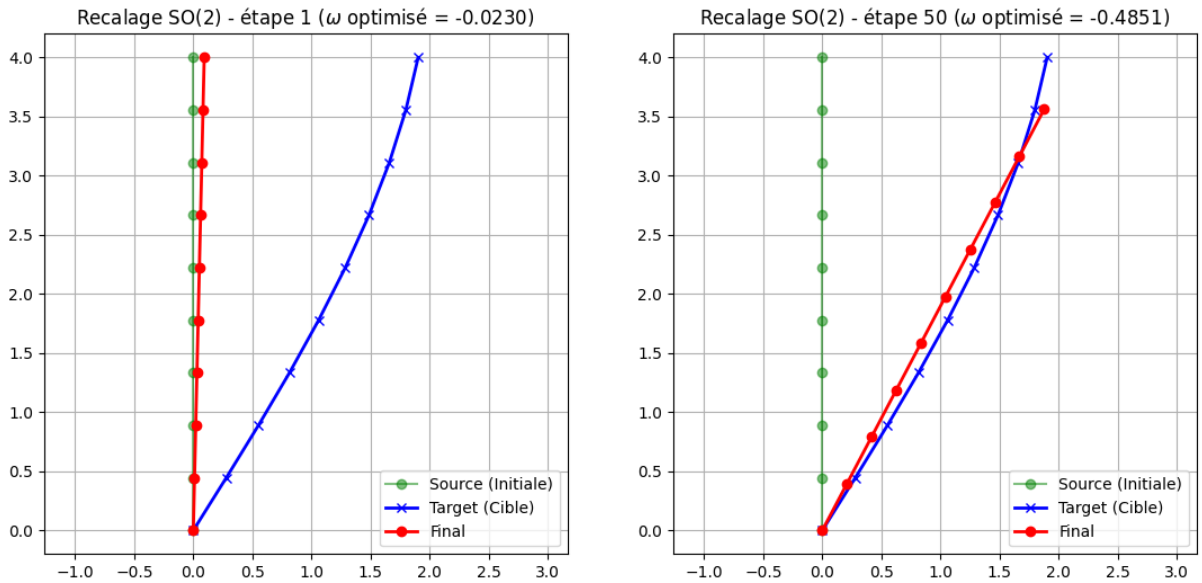


FIGURE 5.1 – Première et 50^{ième} étape du recalage sur les rotations $SO(2)$.

Exemple 5.2 (Recalons Lecocq).

Voyons des exemples de recalage par rotation sur $SO(2)$ avec des formes plus complexes.

Dans la **Figure (5.2)**, on applique une rotation à un coq x_S pour le faire “matcher” avec la forme cible x_T qui est le sensiblement le même coq légèrement tourné. On voit que les deux figures superposées et cible se superposent assez bien. La géométrie du coq est préservée tout au long de la déformation, le cadre LDDMM restreint aux rotations est efficace lorsque la transformation est seulement une rotation.

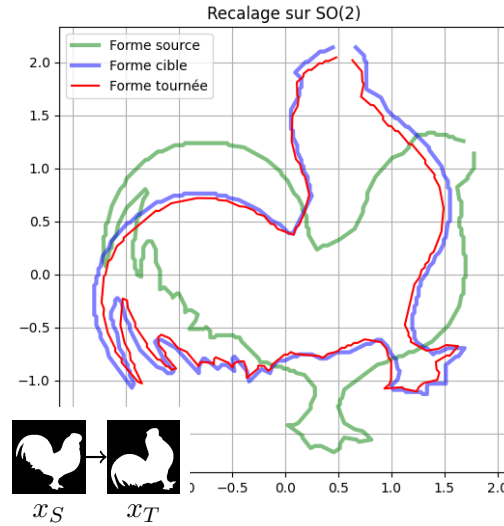


FIGURE 5.2 – Recalage sur les rotations $SO(2)$ d'un coq.

La **Figure (5.3)** présentent deux cas où la forme cible x_T diffère de la forme source x_S non seulement par une rotation, mais aussi par une déformation (des coq légèrement différents séparé de x_S par une transformation non rigide). Dans ces situations, le modèle restreint à $SO(2)$ montre ses limites : en cherchant uniquement une rotation optimale, l'algorithme ne peut pas capturer les déformations non rigides. La forme transformée x_1 ne coïncide alors que partiellement et voir même pas du tout avec la cible x_T , et le terme d'attache aux données $\|x_1 - x_T\|$ reste élevé malgré la convergence de l'optimisation.

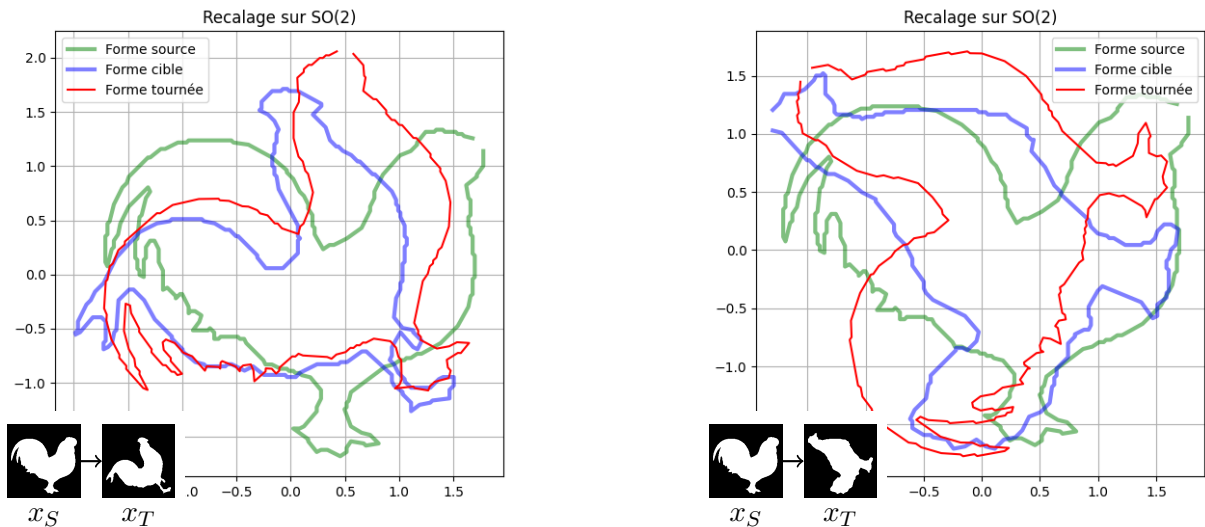


FIGURE 5.3 – Recalage sur les rotations $SO(2)$ de différents coqs.

Conclusion

Références

- [1] Mirza Faisal Beg, Michael Miller, Alain Trouvé, and Laurent Younes. *Computing Large Deformation Metric Mappings via Geodesic Flows of Diffeomorphisms*. International Journal of Computer Vision, 2005.
- [2] Darryl Holm. *Geometric mechanics - Part I : Dynamics and symmetry*. World Scientific, 2011.
- [3] Darryl D. Holm. 31 lectures on geometric mechanics, 2024.
- [4] Rayane Mouhli and Thomas Pierron. *Decoupling actions of finite-dimensional Lie groups and of groups of diffeomorphisms in the large deformation framework*, 2025.
- [5] Thomas Pierron. Right-invariant sub-riemannian geometry in the banach setting and applications to large deformations models in shape analysis. *ESAIM : Proceedings and Surveys*, 2025.
- [6] Thomas Polzin. *Large deformation diffeomorphic metric mappings : Theory, numerics, and applications*. PhD thesis, Zentrale Hochschulbibliothek Lübeck, 2018.
- [7] Laurent Younes. *Shapes and diffeomorphisms*. Springer, 2010.

A Code Python

Tout le code sera disponible sur [ce dépôt](#) GitHub.

```
1 def Euler(q0, omega, nt):
2     dt = 1.0 / (nt - 1)
3     q = q0.clone()
4     A = skew_matrix(omega)
5
6     Q = torch.zeros((q0.shape[0], q0.shape[1], nt), dtype=q0.dtype,
7 device=q0.device)
8     Q[:, :, 0] = q
9
10    for t in range(1, nt):
11        q = q + dt * (q @ A.T)
12        Q[:, :, t] = q
13
14    return q, Q
```

Code A.1 – Fonction Euler

```
1 def rotation_loss(data_loss_fnc, nt):
2     def loss(q0, omega):
3         q_final, _ = Euler(q0, omega, nt)
4         dloss = data_loss_fnc(q_final)
5         reg = 0.5 * (omega ** 2)
6         return reg + dloss
7     return loss
```

Code A.2 – Fonction rotation_loss

```
1 def opti_rotation(q0, loss_fnc, n_iter, eta):
2     omega = torch.zeros(1, dtype=q0.dtype, device=q0.device,
3 requires_grad=True)
4     optimizer = torch.optim.LBFGS([omega], lr=eta, line_search_fn='
5 strong_wolfe')
6     losses = []
7
8     def closure():
9         optimizer.zero_grad()
10        L = loss_fnc(q0, omega)
11        L.backward()
12        return L
13
14    for i in range(n_iter):
15        L = optimizer.step(closure)
16        losses.append(L.item())
17
18    return omega.detach(), losses
```

Code A.3 – Fonction opti_rotation

B Figures supplémentaires

Quelques figures supplémentaires.

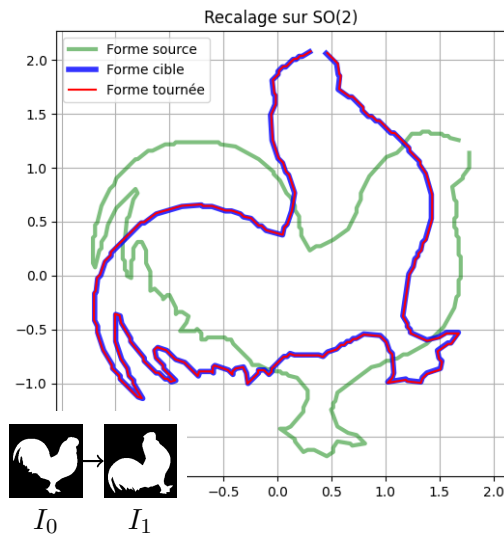


FIGURE B.1 – Complète l'exemple (1.2)

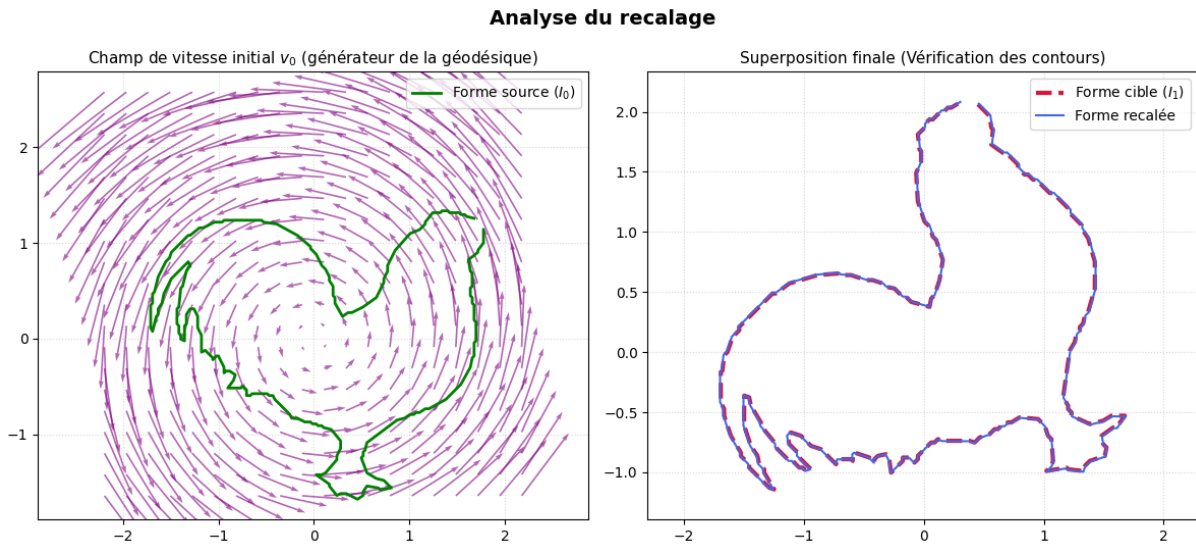


FIGURE B.2 – Complète l'exemple (1.2)

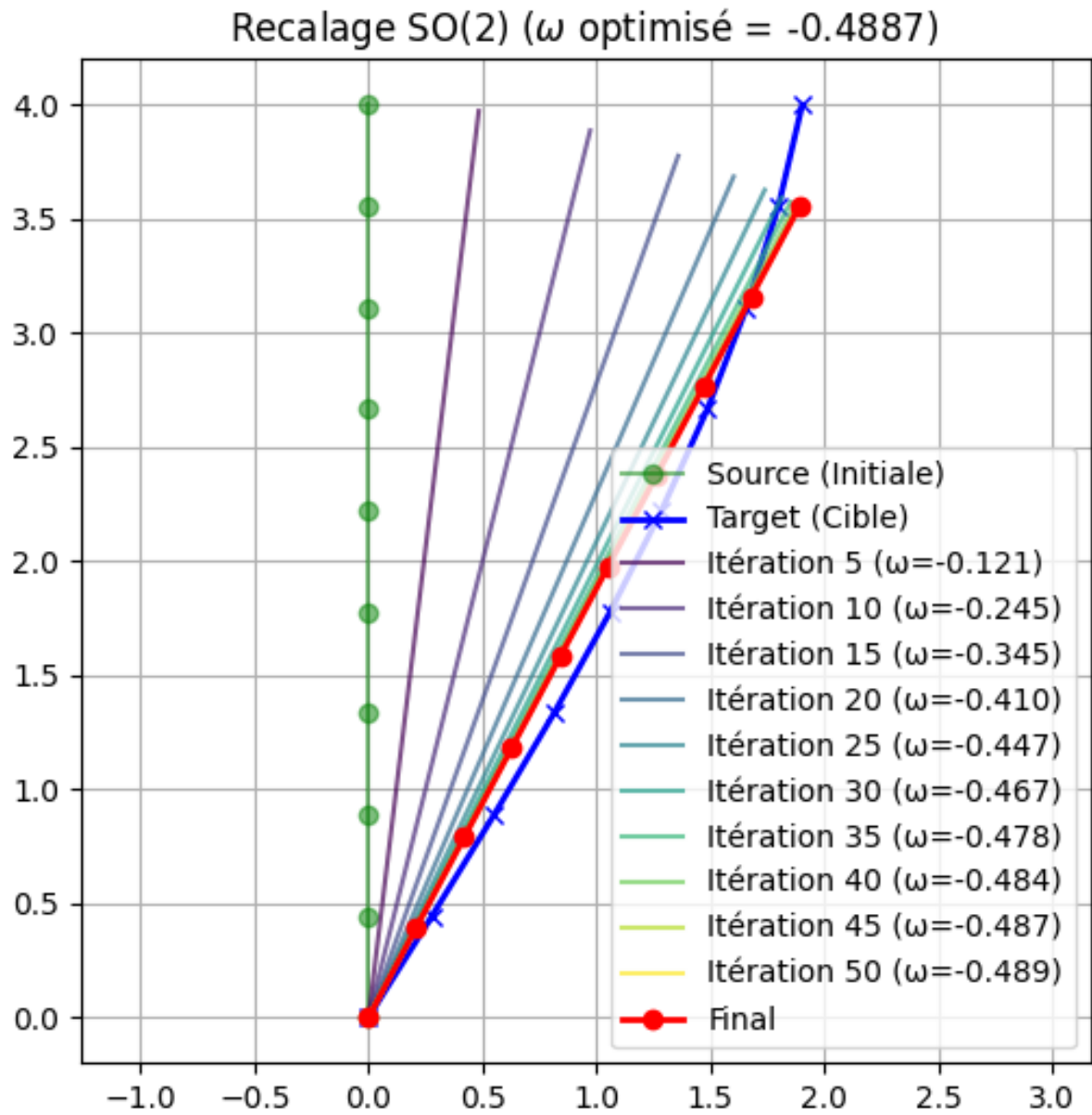


FIGURE B.3 – Complète l'exemple (5.1)

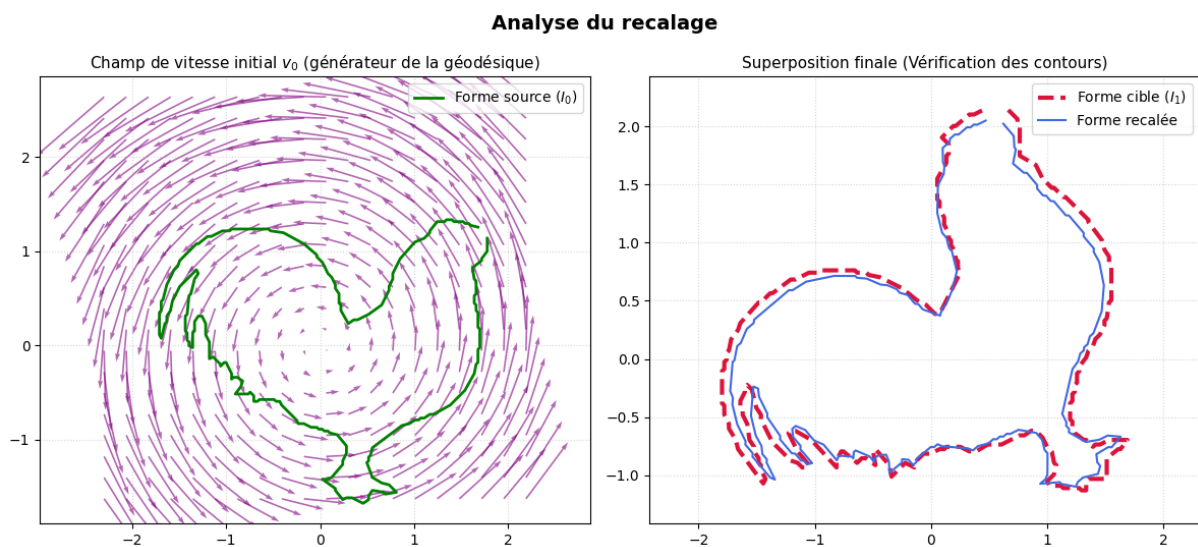


FIGURE B.4 – Complète l'exemple (5.2)

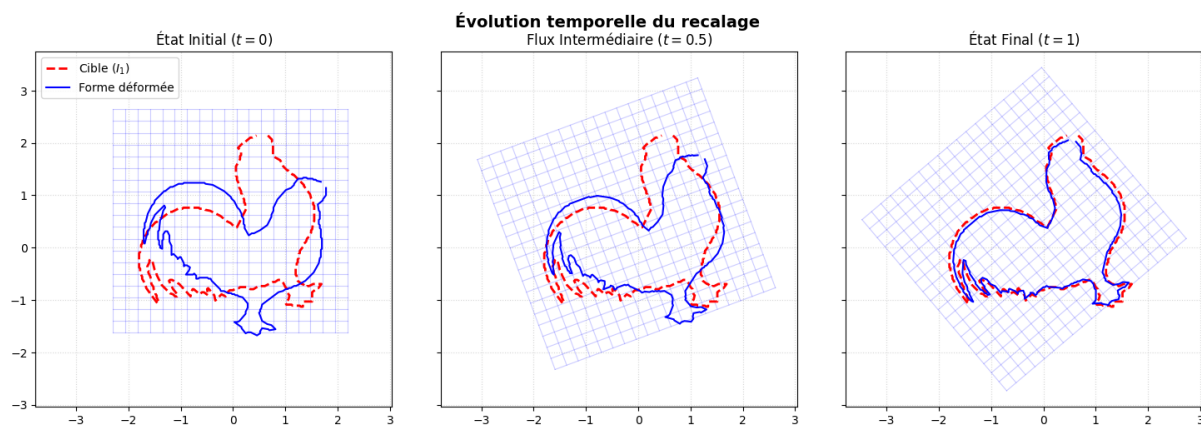


FIGURE B.5 – Complète l'exemple (5.2)

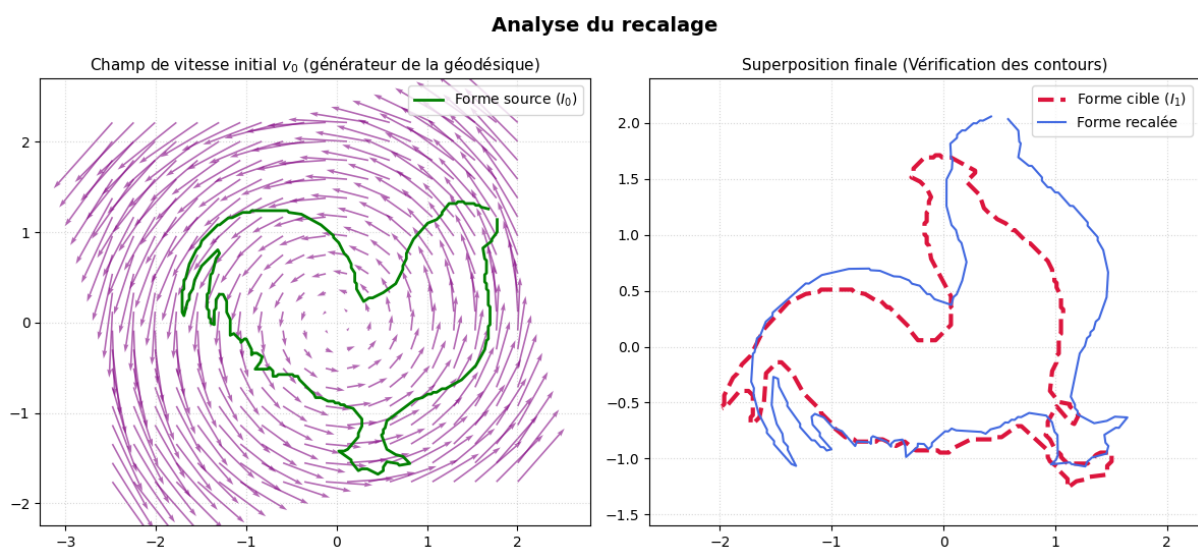


FIGURE B.6 – Complète l'exemple (5.2)

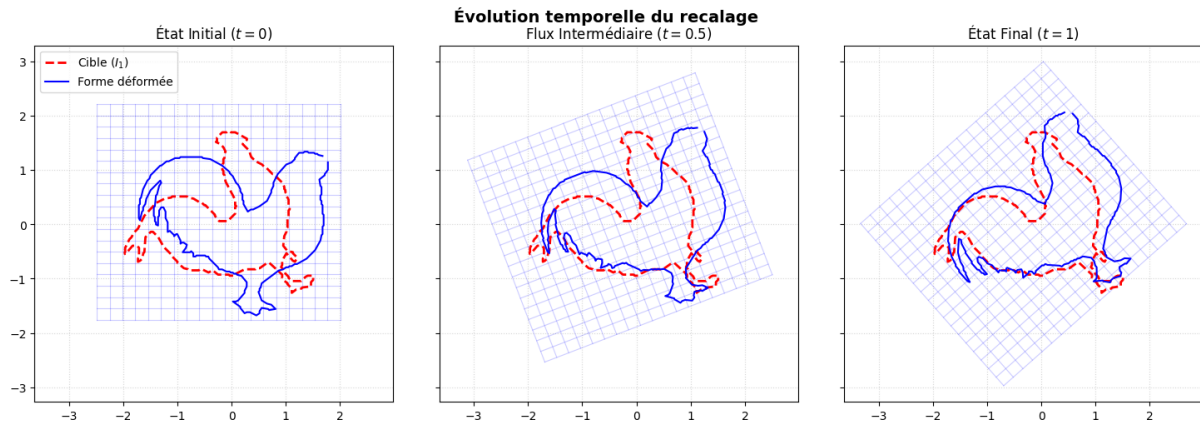


FIGURE B.7 – Complète l'exemple (5.2)

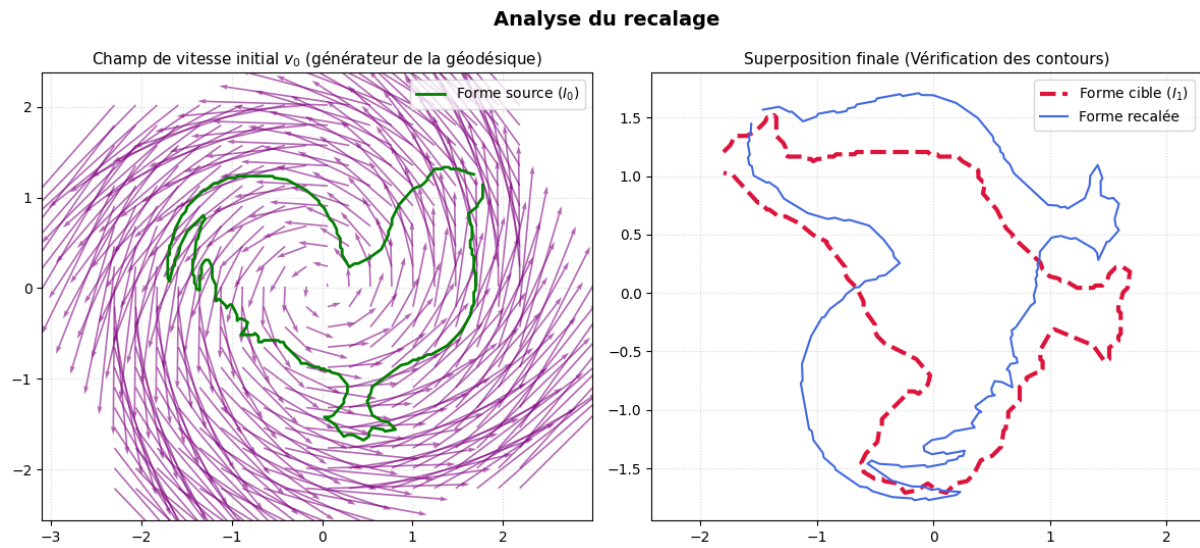


FIGURE B.8 – Complète l'exemple (5.2)

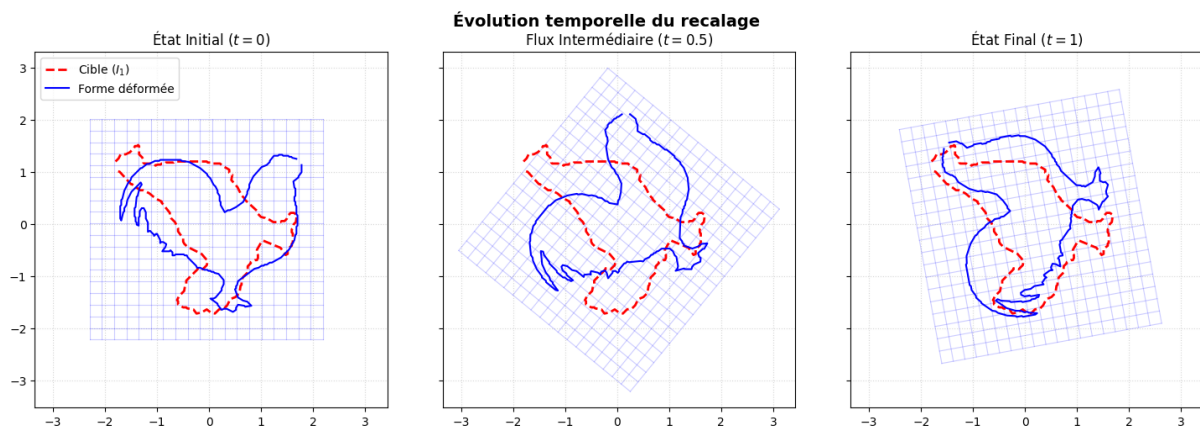


FIGURE B.9 – Complète l'exemple (5.2)